

Deploy a MERN Chat App on AWS EC2

Windows + Ubuntu + MongoDB Atlas + PM2 + Nginx + optional HTTPS

Browser	Nginx	Node/Express	MongoDB Atlas
React frontend	Ports 80/443	localhost:5000	cloud database

Prepared for your current setup

EC2 instance: chat-app-server | Public IPv4: 44.197.248.232 | Windows key file: chat-app-server-key.pem

Security note: the public IP can change after stopping and starting the instance unless an Elastic IP is attached.

1. Prerequisites and project assumptions

- AWS EC2 Ubuntu instance is running.
- Security group permits SSH 22 from your IP, HTTP 80 from anywhere, and HTTPS 443 from anywhere.
- The project is stored in GitHub and contains frontend and backend folders, or equivalent paths.
- MongoDB Atlas connection string is available.
- Backend reads PORT and MONGO_URI from environment variables.

Recommended project shape

```
chat-app/  
backend/  
server.js (or app.js/index.js)  
package.json  
frontend/  
package.json  
src/
```

2. Connect from Windows

The PEM file is downloaded when the EC2 key pair is created. Keep it private and do not commit it to GitHub.

Open PowerShell and move to Downloads:

```
cd C:\Users\40013788\Downloads  
dir *.pem
```

Connect using the Ubuntu user:

```
ssh -i chat-app-server-key.pem ubuntu@44.197.248.232
```

If SSH times out during key exchange

Your checks showed that port 22 is reachable and the EC2 SSH service is active. A corporate VPN, proxy, or endpoint security product may interrupt the SSH handshake. Disconnect the work VPN and retry through a mobile hotspot. AWS EC2 Instance Connect in the browser is a valid fallback.

```
Test-NetConnection 44.197.248.232 -Port 22  
ssh -vvv -i chat-app-server-key.pem ubuntu@44.197.248.232
```

3. Update Ubuntu and install tools

```
sudo apt update && sudo apt upgrade -y  
sudo apt install -y git curl build-essential nginx
```

Verify Nginx:

```
sudo systemctl enable --now nginx  
sudo systemctl status nginx
```

4. Install Node.js with NVM

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash  
source ~/.bashrc  
nvm install --lts  
nvm alias default node  
node -v  
npm -v
```

5. Clone and configure the chat app

For a public repository:

```
cd /home/ubuntu
git clone YOUR_GITHUB_REPOSITORY_URL chat-app
cd chat-app
```

For a private repository, use a GitHub deploy key or a personal access token. Do not place credentials directly in shell history or source files.

6. Configure and test the backend

```
cd /home/ubuntu/chat-app/backend
npm ci --omit=dev
```

Create the production environment file:

```
nano .env

NODE_ENV=production
PORT=5000
MONGO_URI=mongodb+srv://USERNAME:PASSWORD@CLUSTER/chatapp
JWT_SECRET=REPLACE_WITH_A_LONG_RANDOM_SECRET
CLIENT_URL=http://44.197.248.232
```

Save in nano with Ctrl+O, Enter, then Ctrl+X. If the password contains reserved URL characters, URL-encode those characters.

Test the backend using the actual entry file or npm script:

```
npm start
# In another session:
curl http://localhost:5000
```

Stop the temporary test with Ctrl+C after it succeeds.

7. Keep the backend running with PM2

```
sudo npm install -g pm2
cd /home/ubuntu/chat-app/backend
pm2 start npm --name chat-api -- start
pm2 status
pm2 logs chat-api --lines 50
pm2 save
pm2 startup
```

PM2 prints one additional sudo command. Copy and run that exact command, then run pm2 save again.

8. Build the React frontend

```
cd /home/ubuntu/chat-app/frontend
npm ci
npm run build
```

Vite normally creates dist. Create React App normally creates build. Use the directory that your project actually generates.

Prefer a relative API base URL so Nginx can route API calls:

```
// Example
const api = axios.create({ baseURL: "/api" });
```

9. Publish the frontend

For Vite:

```
sudo rm -rf /var/www/chat-app
sudo mkdir -p /var/www/chat-app
sudo cp -r /home/ubuntu/chat-app/frontend/dist/* /var/www/chat-app/
```

For Create React App, replace dist with build.

10. Configure Nginx

```
sudo nano /etc/nginx/sites-available/chat-app
```

```
server {
    listen 80;
    server_name _;

    root /var/www/chat-app;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /socket.io/ {
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
    }
}
```

```
sudo ln -sf /etc/nginx/sites-available/chat-app /etc/nginx/sites-enabled/chat-app
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

Open <http://44.197.248.232> in a browser. Keep backend port 5000 closed in the EC2 security group because Nginx reaches it locally.

11. MongoDB Atlas networking

- Create a database user with a strong password.
- Add the EC2 public IP to Atlas Network Access where possible.
- Avoid 0.0.0.0/0 for long-term use because it permits connection attempts from any IP.
- Restart the API after changing environment variables: `pm2 restart chat-api --update-env`.

12. Optional domain and HTTPS

- Point the domain A record to the EC2 public IP. For a stable address, attach an Elastic IP and review AWS pricing.

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d example.com -d www.example.com
sudo certbot renew --dry-run
```

13. Deploy future updates

```
cd /home/ubuntu/chat-app
git pull
```

Backend update:

```
cd backend
npm ci --omit=dev
pm2 restart chat-api --update-env
pm2 logs chat-api --lines 50
```

Frontend update:

```
cd ../frontend
npm ci
npm run build
sudo rm -rf /var/www/chat-app/*
sudo cp -r dist/* /var/www/chat-app/
sudo nginx -t && sudo systemctl reload nginx
```

14. Troubleshooting checklist

Problem	Check
SSH connection times out	Try a mobile hotspot, disconnect corporate VPN/proxy, use EC2 Instance Connect, and run <code>ssh -vvv</code> .
502 Bad Gateway	Run <code>pm2 status</code> , <code>pm2 logs chat-api</code> , and <code>curl http://127.0.0.1:5000</code> . Confirm Nginx proxy port matches PORT.
React route returns 404	Confirm <code>try_files \$uri \$uri/ /index.html</code> ; exists in the Nginx location / block.
Socket.IO cannot connect	Confirm the <code>/socket.io/</code> block contains Upgrade and Connection headers; verify client and server path/origin settings.
Database connection fails	Check MONGO_URI, Atlas database user, password encoding, and Atlas Network Access.
App stops after reboot	Run <code>pm2 startup</code> , execute the generated sudo command, then <code>pm2 save</code> .

15. Final verification

- `http://PUBLIC_IP` loads the React application.
- API requests to `/api` succeed.
- Real-time chat connects and messages transmit through Socket.IO.
- `pm2 status` shows chat-api online.
- `sudo nginx -t` reports syntax is ok and test is successful.
- After a controlled reboot, Nginx and the PM2 application return automatically.
- Secrets are only in `.env` and `.env` is excluded by `.gitignore`.
- SSH source is restricted to your current public IP once troubleshooting is complete.

Security reminder: Your screenshot currently shows SSH port 22 open to 0.0.0.0/0. After connectivity is resolved, change the SSH security-group source to My IP or another trusted CIDR.

References

Deployment pattern reviewed against current public guidance covering EC2, NVM, PM2, Nginx, SSH, and SSL. URLs are intentionally listed as readable references.

1. ShortIQ - How to Deploy a Node.js App on AWS EC2 with Nginx and PM2
<https://shortiq.io/blog/deploy-nodejs-aws-ec2-nginx-pm2>
2. C# Corner - How to Deploy Node.js App on AWS EC2 with Nginx and PM2
<https://www.c-sharpcorner.com/article/how-to-deploy-node-js-app-on-aws-ec2-with-nginx-and-pm2>